

LAPORAN TUGAS BESAR

Sistem Terdistribusi Dasar

Milestone 1

Platform Koordinasi Kebencanaan BNPB

Kelompok **AkuAnakTehat**

Dzaky Aurelia Fawwaz 13523065

Muhammad Alfansya 13523005

IF4031 Arsitektur Aplikasi Terdistribusi

8 Oktober 2026

Sumber LaTeX dan diagram disertakan dalam repository.

Revisi implementasi acuan **f6e99cc**.

Bukti pengujian lokal bertanggal 6 Oktober 2026.

Kontribusi Anggota

Pembagian berikut menjelaskan kontribusi setiap anggota berdasarkan komponen sistem dan tanggung jawab yang dikerjakan.

Aspek	Dzaky Aurelia Fawwaz (13523065)	Muhammad Alfansya (13523005)
Perancangan	Perancangan arsitektur sistem dan mekanisme penyelesaian P1 hingga P5.	Membantu verifikasi dan validasi kesesuaian rancangan terhadap spesifikasi.
Implementasi	BMKG Mock dan PVMBG Mock; Auth Service; polling, pemetaan, korelasi tsunami, transaksi dan watermark pada Aggregator; outbox relay dan producer Kafka; Dashboard Updater, Notifier, dan Pemda Portal; infrastruktur Docker Compose, bootstrap secret, dan CI; integrasi serta pengujian lintas komponen.	Query dan API internal Aggregator; Client API beserta rate limit, batas konkurensi, pagination, dan penanganan galat; CLI Tim Lapangan; serta script load test k6.
Penulisan laporan	<i>[Isi kontribusi penulisan Dzaky]</i>	<i>[Isi kontribusi penulisan Muhammad Alfansya]</i>

Perlu dilengkapi. Konfirmasi dan isi kontribusi penulisan kedua anggota pada `metadata.tex` sebelum pengumpulan. Tidak ada persentase kontribusi yang diasumsikan.

Laporan disusun dari kode yang telah diintegrasikan, bukti eksekusi lokal, dan rancangan awal kelompok. Bantuan LLM dijelaskan secara eksplisit pada Bagian 11; tanggung jawab pemahaman dan verifikasi akhir berada pada kelompok.

Daftar Isi

Kontribusi Anggota	i
1 Deskripsi Umum dan Lingkup	1
1.1 Prinsip rancangan	1
1.2 Versi yang dilaporkan	1
2 Arsitektur Sistem	2
2.1 Komponen dan kepemilikan data	3
2.2 Protokol dan batas kepercayaan	3
2.3 Alur lintas komponen	4
3 Teknologi dan Asumsi	4
3.1 Teknologi terpilih	4
3.2 Asumsi operasional dan data	4
3.3 Keputusan yang berubah dari rancangan awal	5
4 Petunjuk Menjalankan Sistem	5
5 P1. Arsitektur dan Interoperabilitas Data	6
5.1 Masalah dan mekanisme	6
5.2 Pemetaan kanonik	7
5.3 Deteksi perubahan dan transaksi	7
5.4 Alternatif dan trade-off	9
5.5 Bukti penyelesaian	9
6 P2. Konkurensi, Availability, dan Degradasi	10
6.1 Model konkurensi dan isolasi	10
6.2 Konfigurasi dasar	10
6.3 Kebaruan data dan kontrak galat	12
6.4 Konfigurasi dan hasil pengukuran	12
6.5 Validitas hasil dan alternatif	14
7 P3. Autentikasi dan Domain Kepercayaan	14
7.1 Pemisahan kredensial	14
7.2 JWT, scope, dan proyeksi	14
7.3 Rotasi refresh dan batas atomisitas	15
7.4 Alasan pemilihan dan risiko yang ditangani	17
7.5 Bukti penyelesaian	17

8 P4. Batas Service dan Flexible Storage	17
8.1 Alasan menentukan batas service	17
8.2 Kepemilikan Canonical Store	18
8.3 Skema dan evolusi atribut	18
8.4 Perbandingan alternatif penyimpanan	19
8.5 Bukti independensi dan batas	19
9 P5. Distribusi Event dengan Consumer Independen	20
9.1 Broker dan pola yang dipilih	20
9.2 Outbox dan semantik pengiriman	20
9.3 Efek consumer, idempotensi, dan DLQ	21
9.4 Alternatif dan trade-off	23
9.5 Bukti penyelesaian	23
10 Ringkasan Verifikasi dan Batas Hasil	23
10.1 Kesimpulan dan batas cakupan	24
11 Deklarasi Penggunaan LLM	25
A Matriks Kriteria Spesifikasi	26
B Indeks Bukti dan Jejak Eksekusi	28
B.1 Contoh trace event nyata	28
B.2 Asal potongan kode dan gambar	29
B.3 Dokumentasi demonstrasi	29
Daftar Pustaka	30

1 Deskripsi Umum dan Lingkup

BNPB membutuhkan representasi yang konsisten atas informasi gempa, potensi tsunami, dan aktivitas gunung api dari dua instansi yang otonom. BMKG dan PVMBG memiliki skema, kredensial, serta karakteristik latensi berbeda. Sistem ini mengambil data melalui polling, memetakannya menjadi **HazardEvent**, menyajikan data sesuai hak akses client, dan menyalurkan perubahan melalui broker kepada consumer independen.

Lingkup Milestone 1 mengikuti dokumen spesifikasi M1 dan konteks tugas terpusat [1, 2]. Kedua instansi berupa mock yang dibangun kelompok; data serta referensi gunung api bersifat sintetis. Tiga identitas downstream adalah Media, Tim Lapangan, dan BNPB Pusat. Seluruh akses downstream bersifat baca saja. Pelaporan dari lapangan, integrasi sistem instansi nyata, dan pengoperasian banyak instance tidak termasuk implementasi ini.

1.1 Prinsip rancangan

Sistem memisahkan proses ingest, pembacaan data, dan distribusi event. Proses ingest menangani ketidakpastian sumber, sedangkan pembacaan menggunakan data yang sudah tersimpan. Perubahan data selanjutnya diteruskan melalui outbox dan Kafka. Pemisahan ini membuat satu request client tidak harus menunggu HTTP ke instansi sumber. Hasil pembacaan karena itu dapat tertinggal dari kondisi terbaru di sumber. Waktu data dan status sumber disertakan agar pengguna dapat menilai kebaruannya.

Problem	Mekanisme inti yang diimplementasikan
P1	Tolerant reader JSON, pemetaan kanonik, korelasi warning, dan preservasi field tambahan.
P2	Worker per sumber, baca dari store, timeout, circuit breaker, batas konkurensi, rate limit, dan status stale.
P3	Kredensial per domain, JWT Ed25519, scope, proyeksi allowlist, dan rotasi refresh token.
P4	Container dan module terpisah, kepemilikan storage, serta PostgreSQL dengan atribut JSONB.
P5	Transactional outbox, Kafka, group consumer terpisah, SQLite persisten, dedup, dan DLQ.

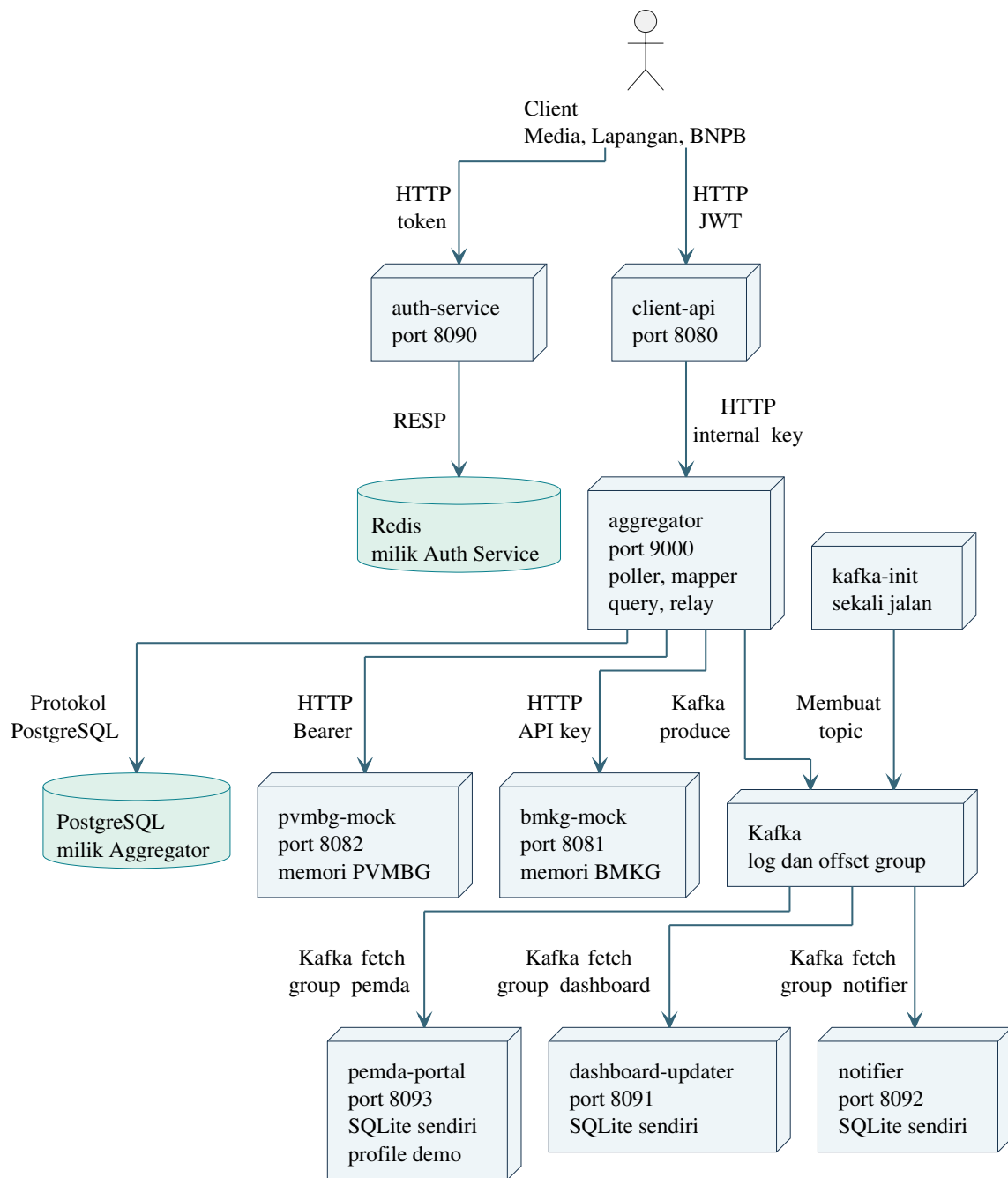
1.2 Versi yang dilaporkan

Laporan ini mendeskripsikan implementasi acuan `f6e99cc`, bukan seluruh fitur pada rancangan awal. Bukti lokal bertanggal 6 Oktober 2026. Singleflight dan micro-cache, `LISTEN/NOTIFY`, tabel `schema_observations`, dan propagasi deadline melalui header belum diimplementasikan. Timeout lokal, relay polling, dan log `schema_drift` sudah menjadi bagian inti.

Tautan kode dan bukti pada laporan menunjuk revisi acuan tersebut agar klaim dapat ditelusuri meskipun branch utama berubah. Perubahan laporan setelah revisi itu tidak dengan sendirinya berarti pengujian runtime dijalankan ulang.

2 Arsitektur Sistem

Gambar 1 memperlihatkan container, protokol, dan kepemilikan storage sistem.



Gambar 1: Arsitektur sistem beserta protokol dan kepemilikan penyimpanan. Setiap kotak dan simbol database mewakili satu container. Memori mock dan SQLite berada di dalam container pemilik. Panah dari Kafka menunjukkan aliran data menuju consumer yang menginisiasi fetch.

2.1 Komponen dan kepemilikan data

Sistem memiliki delapan service aplikasi apabila pemda-portal diaktifkan. PostgreSQL, Redis, dan Kafka merupakan container infrastruktur. Konfigurasi dasar menjalankan sepuluh container jangka panjang, ditambah satu `kafka-init` yang selesai setelah membuat topic; profile demo menambah pemda-portal. CLI dan load generator adalah client pengujian, bukan service penyimpanan data.

Komponen	Tanggung jawab	Storage dan port
<code>bmkg-mock</code>	Menyajikan gempa dan warning terpisah; seed dan generator; memvalidasi API key BMKG.	Memori sendiri; 8081.
<code>pvmkg-mock</code>	Laporan vulkanik, delay, perubahan skema, dan outage yang dapat dipicu operator.	Memori sendiri; 8082.
<code>aggregator</code>	Polling, tolerant reader, pemetaan, korelasi, transaksi, query internal, dan relay outbox.	Pemilik PostgreSQL; 9000 internal.
<code>client-api</code>	Verifikasi JWT, scope, proyeksi, filter, pagination, dan proteksi beban.	Tanpa store persisten; 8080.
<code>auth-service</code>	Penerbit access token dan refresh token dan pengelola rotasi sesi.	Pemilik Redis; 8090.
<code>dashboard-updater</code>	View event terbaru per hazard, hanya menerima versi yang lebih baru.	SQLite sendiri; 8091.
<code>notifier</code>	Simulasi pengiriman alert SIAGA atau AWAS; dedup persisten.	SQLite sendiri; 8092.
<code>pemda-portal</code>	Subscriber tambahan untuk menunjukkan perluasan stream tanpa mengubah producer.	SQLite sendiri; 8093.

Port host aplikasi dipublikasikan pada loopback `127.0.0.1`. Port database, Redis, Kafka, dan API internal Aggregator tidak dipublikasikan ke host pada konfigurasi normal. Container memakai named volume untuk store persisten; restart biasa tidak menghapus state.

2.2 Protokol dan batas kepercayaan

Aggregator mengambil data JSON melalui HTTP dengan kredensial yang berbeda untuk setiap instansi. BMKG memerlukan `X-BMKG-Key`, sedangkan PVMBG memerlukan Bearer token miliknya sendiri. Client menukar kredensial ke `/oauth/token`, kemudian mengirim JWT ke `client-api`. Client-api membaca lewat HTTP internal Aggregator dengan `X-Internal-Key`; ia tidak mempunyai kredensial PostgreSQL. Auth-service mengakses Redis melalui RESP dan password yang berbeda.

Relay mengirim envelope melalui protokol Kafka. Consumer mengambil stream dengan group masing-masing dan melakukan efek lokal di SQLite. Tidak ada pemanggilan fungsi maupun shared business package lintas service. Kontrak yang sama direpresentasikan secara lokal di setiap module. Komponen dalam Aggregator tetap boleh berkomunikasi lewat port berupa interface Go karena berada pada service pemilik yang sama.

Network `store_net` menghubungkan Aggregator dan PostgreSQL; `edge_net` menghubungkan

client-api dan Aggregator; **source_net** menghubungkan sumber dan poller; **auth_net** menghubungkan auth-service dan Redis; **bus_net** menghubungkan producer, broker, dan consumer. Profile test mempunyai akses yang diperlukan sebagai perangkat uji administratif, bukan jalur akses aplikasi produksi.

Implementasi terkait terdapat pada [docker-compose.yml](#).

2.3 Alur lintas komponen

Pada ingest, satu batch valid menghasilkan perubahan HazardEvent, outbox, dan checkpoint dalam satu transaksi PostgreSQL. Pada baca, client-api menerapkan autentikasi dan batas beban sebelum mengakses query internal; hasil kemudian diproyeksikan menurut scope. Pada distribusi, relay memublikasikan snapshot yang sudah committed, lalu menandai outbox setelah ACK. Ketiga alur diuraikan lebih lanjut pada P1, P2, P3, dan P5.

3 Teknologi dan Asumsi

3.1 Teknologi terpilih

Teknologi	Alasan terkait kebutuhan
Go 1.24.2	Goroutine dan context mendukung worker independen serta pembatasan waktu; module per service memisahkan dependensi build.
HTTP dengan JSON, net/http	Kontrak mudah diperiksa dan mendukung field aditif dari mock; biaya serialisasi diterima pada skala M1.
PostgreSQL 16.8, pgx	Transaksi menyatukan hazard, outbox, dan checkpoint; kolom bertipe dipadukan dengan JSONB untuk atribut sumber.
Redis 7.4.2, AOF, Lua	TTL untuk sesi dan eksekusi rotasi atomik terhadap state Redis. Redis bukan penyimpan private key.
JWT dengan EdDSA dan Ed25519	Client-api memverifikasi dengan public key tanpa panggilan auth-service pada setiap request.
Kafka 3.9.1, franz-go	Log persisten dan offset per group memungkinkan replay serta subscriber baru yang independen.
SQLite melalui modernc	View dan dedup lokal consumer tanpa menambah server database untuk setiap consumer.
Docker Compose	Container, network, volume, health check, dan konfigurasi tersusun dalam satu orkestrasi.
golang-migrate	Migrasi DDL berversi, di-embed dan dijalankan Aggregator; field aditif JSONB tidak memicu migrasi.
k6 0.57.0	Pengukuran beban berkelanjutan, percentile, throughput, dan error dengan denominator eksplisit.

3.2 Asumsi operasional dan data

1. Satu host Compose, satu instance per service, satu relay berurutan, dan satu broker Kafka. Konfigurasi ini bukan high availability.

2. Jaringan demo dianggap lingkungan lokal tepercaya. HTTP internal tidak memakai TLS dan Kafka tidak memakai SASL. Isolasi network dan kredensial mencegah akses aplikasi yang tidak semestinya dalam model ini, tetapi tidak melindungi dari administrator host yang berbahaya.
3. ID logis sumber tetap saat record yang sama dipoll ulang. Hazard disatukan melalui pasangan (`source`, `source_ref_id`). Stabilitas ID kanonik bergantung pada keberlangsungan Canonical Store.
4. Mock memakai seed minimal 20 record, generator default setiap 10 detik, dan memori lokal. Restart mock dapat kehilangan data runtime yang belum diambil; mekanisme ini bukan jaminan histori instansi produksi.
5. `since` gempa dan laporan bersifat inklusif menurut waktu kejadian atau laporan. Warning tidak memiliki timestamp perubahan pada kontrak wire; mock menyimpan waktu perubahan secara internal untuk filtering. Kompatibilitas dengan mock lain memerlukan interpretasi `since` warning yang sama.
6. Watermark memakai waktu mulai request Aggregator dengan overlap 10 detik. Ini mengasumsikan jam dan keterlambatan sumber berada dalam batas yang wajar; data bertimestamp lama di luar overlap dapat terlewat. Tidak ada jaminan toleransi keterlambatan tak terbatas.
7. Tabel referensi gunung api tersedia secara statis dan sintetis. `volcano_id` yang tidak dikenal dikarantina, bukan diberi koordinat tebakan.
8. Clock yang cukup selaras diperlukan untuk JWT dan timestamp. Implementasi verifikator tidak menambahkan leeway 5 detik yang pernah disebut rancangan awal. Anomali pengukuran clock load generator dibahas pada P2.

3.3 Keputusan yang berubah dari rancangan awal

Versi implementasi memakai log `schema_drift`, bukan tabel pengamatan skema. Notifier dan pemda mempunyai endpoint inspeksi loopback 8092 dan 8093. Idempotensi notifikasi dibatasi oleh celah pengiriman sebelum marker persisten. Rotasi Redis tidak menjamin respons HTTP token baru sampai ke client. Urutan Kafka hanya berlaku dalam partisi dengan pemetaan key yang tetap. Perubahan ini dipertahankan sebagai batas rancangan, bukan ditutupi sebagai jaminan yang sudah terpenuhi.

4 Petunjuk Menjalankan Sistem

Prasyarat aplikasi adalah Go 1.24.2 dan Docker dengan Compose v2. Python 3 diperlukan untuk runner load dan helper trace. Perintah berikut dijalankan melalui PowerShell dari direktori utama repository.

```
powershell -NoProfile -File scripts/secrets/generate.ps1
docker compose config --quiet
docker compose up -d --build --wait --wait-timeout 180
```

Generator menyiapkan kredensial berbeda, pasangan key JWT, dan berkas environment lokal yang diabaikan Git. Nilai rahasia tidak disalin ke laporan. Pada POSIX, `make up` menyediakan bootstrap dan start. Pemda Portal dapat ditambahkan melalui perintah berikut.

```
docker compose --profile demo up -d --build pemda-portal
```

Pengujian unit dan analisis statis dengan `go vet` dijalankan melalui `scripts/check/check.ps1` atau `make check`. Pengujian runtime harus berurutan karena memicu perubahan skema, outage, dan restart dependensi. Rangkaian regresi lengkap dijalankan dengan perintah berikut.

```
$env:GOWORK='off'
go test ./scripts/check/foundation_test.go '
./scripts/check/ingest_test.go ./scripts/check/events_test.go '
./scripts/check/query_test.go -v -count=1 -timeout=15m
py scripts/loadtest/run.py
```

CLI dijalankan dari module `tools/field-cli`; kredensial dibaca dari environment lokal. Perintah berikut menjalankan sesi yang melewati masa berlaku access token.

```
go run ./cmd/field-cli list --raw --limit 1 --watch 65s --count 2
```

Gunakan identitas Tim Lapangan untuk raw. Endpoint `/health` menyatakan proses hidup; endpoint `/ready` aplikasi memeriksa dependensi yang relevan. Status proses hidup saja tidak membuktikan bahwa data semua sumber masih baru. Jejak satu alur dapat ditelusuri menggunakan correlation ID melalui perintah berikut.

```
py scripts/trace/trace.py <correlation-id>
```

Rincian endpoint, konfigurasi, cara memuat kredensial, dan skenario ada pada [README.md](#), [infra/README.md](#), [scripts/check/README.md](#), serta [scripts/loadtest/README.md](#). Penghentian dan pengaktifan kembali secara normal mempertahankan volume; menghapus volume tidak dipakai sebagai bukti recovery.

5 P1. Arsitektur dan Interoperabilitas Data

5.1 Masalah dan mekanisme

Kontrak inti sumber diketahui, tetapi field tambahan dapat muncul saat runtime. Karena itu parser memisahkan validasi field wajib dari penanganan field lain. Respons dibaca sebagai array `json.RawMessage`; setiap record diubah menjadi map nilai JSON. Field wajib diambil dan diperiksa tipe dan rentangnya. Field yang tersisa menjadi **Extra**, kemudian diteruskan ke `attributes` tanpa dikonversi menjadi float64 secara umum.

Kebijakan yang dipilih adalah **mempertahankan field tak dikenal yang valid**. Ini mencakup `confidence_level`, objek bersarang, array, boolean, dan angka dalam rentang yang dapat disimpan JSONB. Nama sumber yang bertabrakan dengan atribut buatan BNPB, misalnya `tsunami_warnings`, ditempatkan pada namespace `source_extensions`. Poller mencatat nama field baru dalam log `schema_drift`. Field wajib yang hilang atau salah tipe, enum tak dikenal, referensi gunung yang tidak tersedia, atau nilai yang tidak dapat disimpan dikarantina dengan alasan yang eksplisit.

Record invalid dalam array tidak harus menggagalkan record lain. Akan tetapi, respons yang bukan JSON array valid menggagalkan fetch batch; watermark tidak dimajukan untuk fetch yang gagal. Penyimpanan karantina memakai pembungkus teks JSON mentah agar input yang tidak representabel sebagai JSONB masih dapat dicatat.

Implementasi terkait terdapat pada [services/aggregator/internal/application/canonicalize/input.go](#).

5.2 Pemetaan kanonik

Field dan aturan	BMKG	PVMBG
Identitas sumber	<code>event_id</code> menjadi <code>source_ref_id</code> .	<code>report_id</code> menjadi <code>source_ref_id</code> .
Jenis dan area	SEISMIC; <code>region_name</code> .	VOLCANIC; nama pada referensi <code>volcano_id</code> .
Lokasi dan waktu	Koordinat episentrum; <code>occurred_at</code> .	Koordinat referensi BNPB; <code>reported_at</code> .
Severity tanpa warning	NORMAL jika $M < 5$; WASPADA jika $5 \leq M < 6,5$; SIAGA jika $M \geq 6,5$.	Pemetaan langsung Normal, Waspada, Siaga, dan Awas ke enum huruf besar.
Severity dengan warning	Untuk gempa berpotensi tsunami, ikuti tingkat warning terkait; ambil tingkat tertinggi bila lebih dari satu.	Tidak berlaku.
Atribut sumber	Magnitude, kedalaman, potensi tsunami, warning terkait, serta field tambahan.	ID gunung, alert asli, jumlah erupsi, tinggi abu, serta field tambahan.

Aggregator membangkitkan UUID saat hazard pertama dibuat. Upsert mempertahankan identitas itu melalui unique constraint sumber dan ID sumber. `ingested_at` merekam penerimaan pertama, bukan waktu pembacaan terakhir; perubahan berikutnya dicatat dalam metadata internal seperti `updated_at` dan `version`.

Warning disimpan terpisah sehingga boleh mendahului gempa. Ketika gempa tiba, warning yang sudah ada digunakan; ketika warning datang belakangan, hazard terkait dikorelasikan ulang. `related_event_id` yang berubah untuk satu warning ditolak. Hasil warning dan zona diurutkan untuk menghindari perubahan hash hanya karena urutan array yang tidak bermakna.

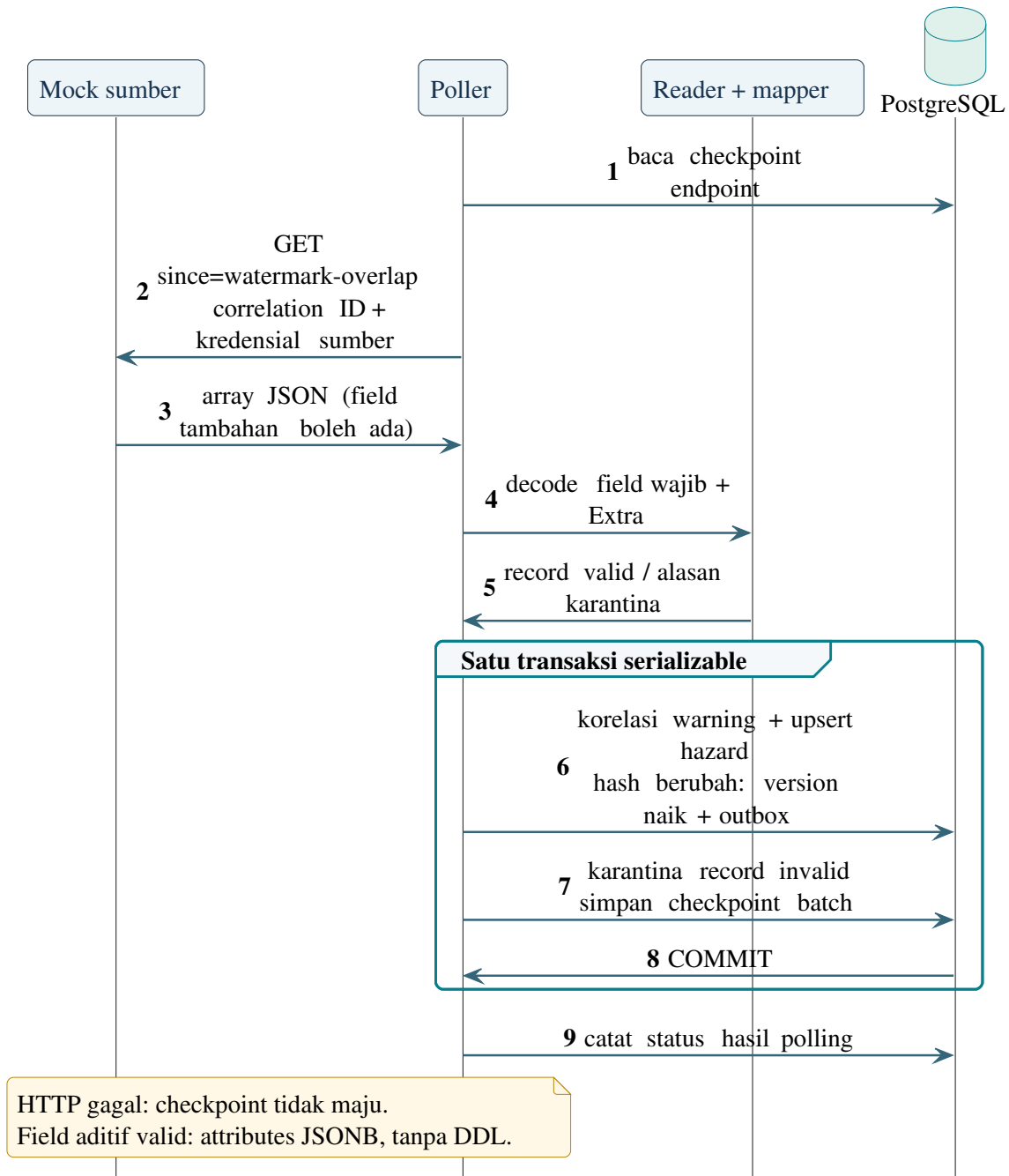
Implementasi terkait terdapat pada [services/aggregator/internal/application/canonicalize/mapper.go](#).

5.3 Deteksi perubahan dan transaksi

Hash konten dihitung dari makna field kanonik dan atribut; ID BNPB serta timestamp penerimaan tidak menjadi alasan perubahan baru. Angka dan timestamp dinormalisasi agar hasil stabil setelah round-trip penyimpanan. Record yang sama tetap memperbarui informasi pengamatan internal tanpa menghasilkan event baru. Perubahan konten meningkatkan `version` dan menghasilkan snapshot outbox dengan event ID baru.

Unit of work memakai transaksi PostgreSQL serializable. Upsert hazard, penyimpanan warning dan record karantina yang relevan, outbox, dan checkpoint batch dilakukan bersama. Rollback transaksi tidak boleh meninggalkan checkpoint yang mengklaim data sudah tersimpan. Ini memindahkan persoalan kegagalan Kafka ke relay tanpa mengorbankan konsistensi ingest.

Implementasi terkait terdapat pada [services/aggregator/internal/application/ingest/service.go](#).



Gambar 2: Ingest dan penambahan field sumber. Kotak transaksi mencakup state yang harus berubah bersama; tidak mencakup publish Kafka.

```

1 func MapVolcanic(v VolcanicInput, reference map[string]Volcano) (hazard.Event, error) {
2     r, ok := reference[v.VolcanoID]
3     if !ok {
4         return hazard.Event{}, errors.New("unknown volcano_id")
5     }
6     a := extras(v.Extra)
7     a["volcano_id"] = raw(v.VolcanoID)
8     a["alert_level"] = raw(v.Alert)
9     a["eruption_count_24h"] = raw(v.Eruptions)
10    a["ash_column_height_m"] = raw(v.Ash)
11    return hazard.Event{Source: "PVMBG", SourceRefID: v.ID, Type: "VOLCANIC", Severity: tsunami.Severity(v.Alert), Area: r.Name, Latitude: r.Latitude, Longitude: r.Longitude, OccurredAt: v.ReportedAt, Attributes: a}, nil
12 }

```

Listing 1: Pemilihan atribut tambahan pada pemetaan vulkanik; kutipan langsung kode implementasi.

5.4 Alternatif dan trade-off

Alternatif	Manfaat	Konsekuensi dan keputusan
Struct ketat dan tolak unknown field	Kontrak sangat eksplisit.	Penambahan aditif memaksa perubahan gateway; tidak dipilih.
JSON tolerant reader	Sesuai wire sumber dan mempertahankan field baru.	Perlu validasi runtime, kebijakan konflik nama, serta batas tipe; dipilih.
Schema registry dengan Avro atau Protobuf	Evolusi skema dapat dikelola dengan aturan formal.	Memerlukan pengelolaan kontrak tambahan dan tidak menggantikan kewajiban menerima wire JSON mock; ditunda.

Yang didukung ialah penambahan field valid tanpa mengubah field wajib. Rename, penghapusan field wajib, perubahan tipe atau arti field, dan enum baru tidak otomatis didukung. Mempertahankan JSON juga tidak berarti menafsirkan semantik field baru secara otomatis. `confidence_level` tersimpan sebagai atribut tambahan; laporan ini tidak mengklaim adanya model kepercayaan yang memakai nilainya untuk menghitung severity.

5.5 Bukti penyelesaian

`TestIngestPipeline` memicu schema v2 dan menemukan `confidence_level` pada Canonical Store serta log drift. `TestDynamicSchemaAndStaleAPI` memeriksa record skema lama dan baru melalui API, versi migrasi tidak berubah, serta container terkait tidak diganti. Pemetaan, field tambahan, korelasi, dan hash diperiksa pada unit test mapper. Rincian kriteria dan lokasi bukti terdapat pada Lampiran A.

Bukti eksekusi tersedia pada [docs/evidence/integration/regression.txt](#).

Implementasi terkait terdapat pada [services/aggregator/internal/application/canonicalize/mapper_test.go](#).

6 P2. Konkurensi, Availability, dan Degradasi

6.1 Model konkurensi dan isolasi

Go menjalankan worker BMKG dan PVMBG secara independen. Dua endpoint BMKG di-fetch paralel, lalu hasilnya diterapkan secara berurutan dalam worker BMKG. Worker tidak memulai siklus berikutnya sebelum siklus aktif selesai; konkurensi fetch tidak tumbuh tanpa batas. Tiap endpoint mempunyai state circuit breaker sendiri.

Jalur baca client mengambil data dari Canonical Store melalui query internal Aggregator, sehingga tidak ikut menunggu PVMBG. Pool PostgreSQL query dipisahkan dari pool ingest. Pembatasan pool membatasi kompetisi koneksi, tetapi keduanya masih memakai proses dan database yang sama. Pemakaian CPU, disk, dan lock database karena itu belum sepenuhnya terisolasi.

Kegagalan	Mekanisme dan batasnya
Head-of-line blocking	Worker sumber terpisah dan materialisasi data mengeluarkan HTTP sumber dari jalur request client. Read tidak menjamin data terbaru sensor.
Resource exhaustion	Semaphore request aktif, token bucket per identitas, pagination, pool koneksi terbatas, dan timeout. Penolakan terkendali dilaporkan sebagai 429, bukan disamarkan sebagai sukses.
Tidak ada degradasi	Status per endpoint diagregasi menjadi status sumber. Data lama tetap tersedia dengan <code>stale_since</code> ; sumber tanpa data usable dapat menghasilkan 503 eksplisit.

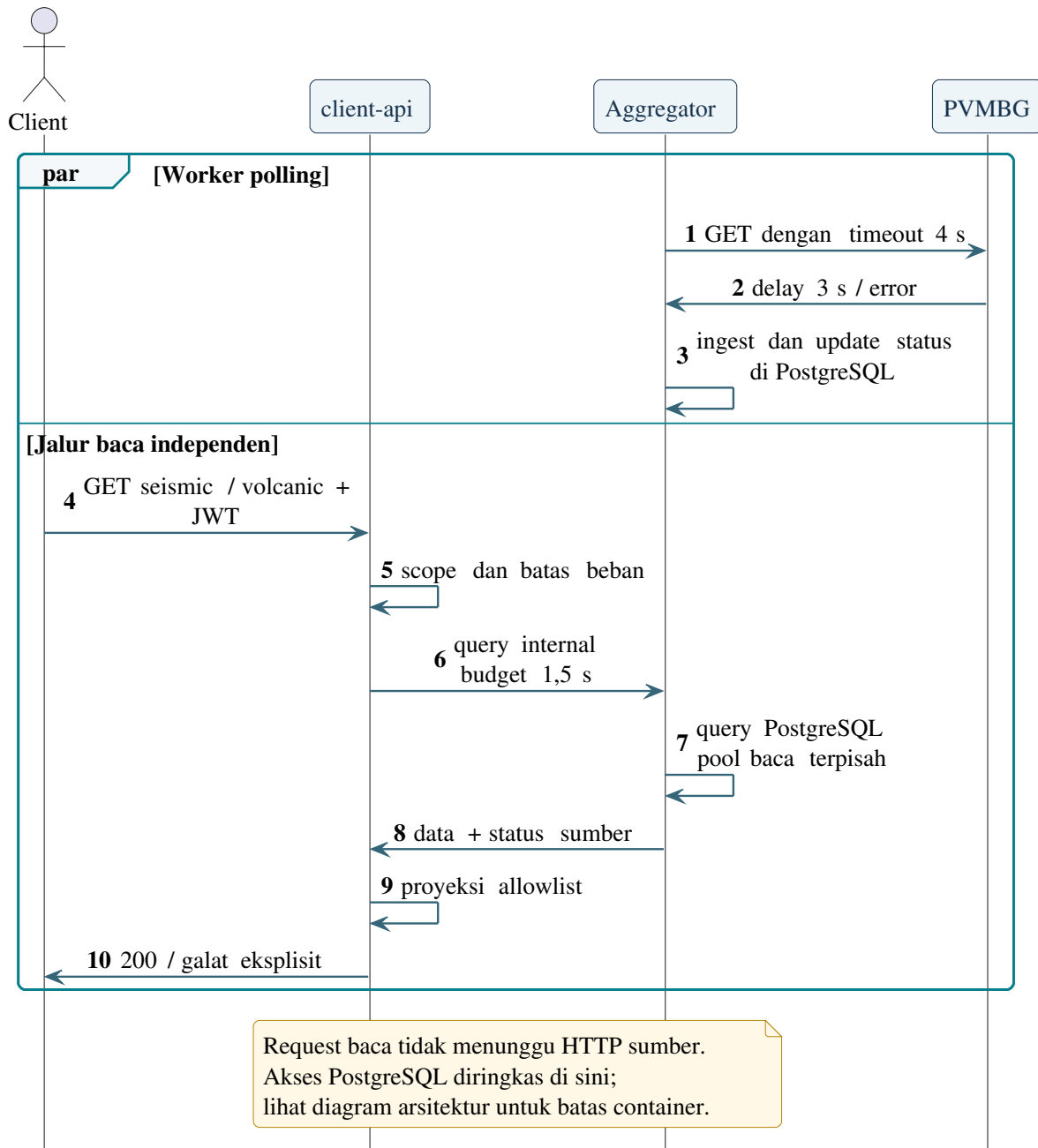
6.2 Konfigurasi dasar

Parameter	Nilai default yang dilaporkan
Interval polling BMKG dan PVMBG	BMKG 2 s dan PVMBG 5 s; jitter kurang dari 10% interval.
Overlap watermark	10 s per endpoint.
Timeout HTTP sumber BMKG dan PVMBG	BMKG 1 s dan PVMBG 4 s.
Timeout operasi dan transaksi PostgreSQL	2 s; deadline parent yang lebih pendek tetap berlaku.
Timeout client-api ke Aggregator	Maksimum 1,5 s total.
Breaker	Buka setelah 3 kegagalan fetch; cooldown 10 s.
Pool PostgreSQL ingest dan query	Ingest 5 koneksi dan query 3 koneksi.
Request aktif client-api	Maksimum 100.
Token bucket per identitas	100 request per detik; kapasitas burst 200.
Ukuran halaman	Default 100, maksimum 500.

Implementasi terkait terdapat pada [services/aggregator/internal/config/config.go](#).

Implementasi terkait terdapat pada [services/client-api/internal/config/config.go](#).

Breaker menahan fetch baru selama sumber berulang kali gagal. Poll yang dilewati bukan percobaan HTTP baru. Saat cooldown selesai, percobaan yang berhasil memulihkan alur tanpa restart. Status sumber sehat baru dapat diketahui setelah polling dan pembaruan status ber-



Gambar 3: Pemisahan request baca dan polling sumber. Pembacaan seismik tetap menggunakan store ketika panggilan PVMBG lambat atau gagal.

ikutnya; mematikan outage tidak membuat metadata langsung HEALTHY pada detik yang sama.

6.3 Kebaruan data dan kontrak galat

`stale_since` menunjukkan awal masalah yang terdeteksi, bukan waktu persis instansi mulai mati. Pada kondisi normal, perkiraan lag mencakup waktu menunggu polling, jitter, durasi fetch, dan transaksi penyimpanan. Secara konseptual,

$$L_{\text{ingest}} \approx W_{\text{poll}} + J + T_{\text{HTTP}} + T_{\text{tx}}.$$

Perkiraan tersebut tidak menjadi jaminan batas waktu layanan. Siklus yang panjang, kegagalan database, ketidakselarasan jam, atau circuit breaker dapat menambah waktu ingest. Outage berkepanjangan membuat umur data terus bertambah. Untuk jalur consumer, tambahkan waktu tunggu relay, ACK broker, dan pemrosesan consumer. Klaim “maksimal satu interval polling” tidak digunakan.

Filter tidak valid menghasilkan 400, identitas atau token invalid 401, akses field terlarang 403, detail yang tidak ditemukan 404, beban ditolak 429, dan dependensi tidak tersedia 503. Hasil filter kosong yang sah tetap 200 dengan array kosong. Error tidak membocorkan detail SQL atau secret.

6.4 Konfigurasi dan hasil pengukuran

Pengukuran akhir dilakukan pada 6 Oktober 2026 menggunakan k6 0.57.0 native Windows. Host memakai AMD Ryzen 9 6900HS, 16 logical processor, dan RAM 16.388.050.944 byte; Docker Desktop 28.0.4 menyediakan 16 CPU dan 7.936.237.568 byte RAM untuk lingkungan container. Tool load dibangun dengan Go 1.23.6; service memakai Go 1.24.2. Seluruh angka diambil dari artefak JSON, bukan angka target yang diasumsikan tercapai.

PVMBG diatur dengan delay minimum dan maksimum 3 s. Skenario paralel menjalankan 10 VU seismic dan 10 VU volcanic selama 60 s. Skenario sustained menjalankan 50 VU selama 90 s dengan jeda 0,1 s per iterasi; setiap VU memiliki sesi refresh sendiri. Ini model closed loop. Semua VU menggunakan identitas Tim Lapangan yang sama, sehingga berbagi rate limit client. Skenario outage memakai 5 VU per fase (20 s outage, 30 s recovery) dan satu VU kontrol.

Skenario	Bisnis/s	Sukses/s	p50 (ms)	p95 (ms)	p99 (ms)
seismic-only	191,47	109,44	5,06	8,41	10,52
sustained	473,04	107,54	5,89	12,49	22,82
outage	23,01	23,01	5,57	14,60	36,02

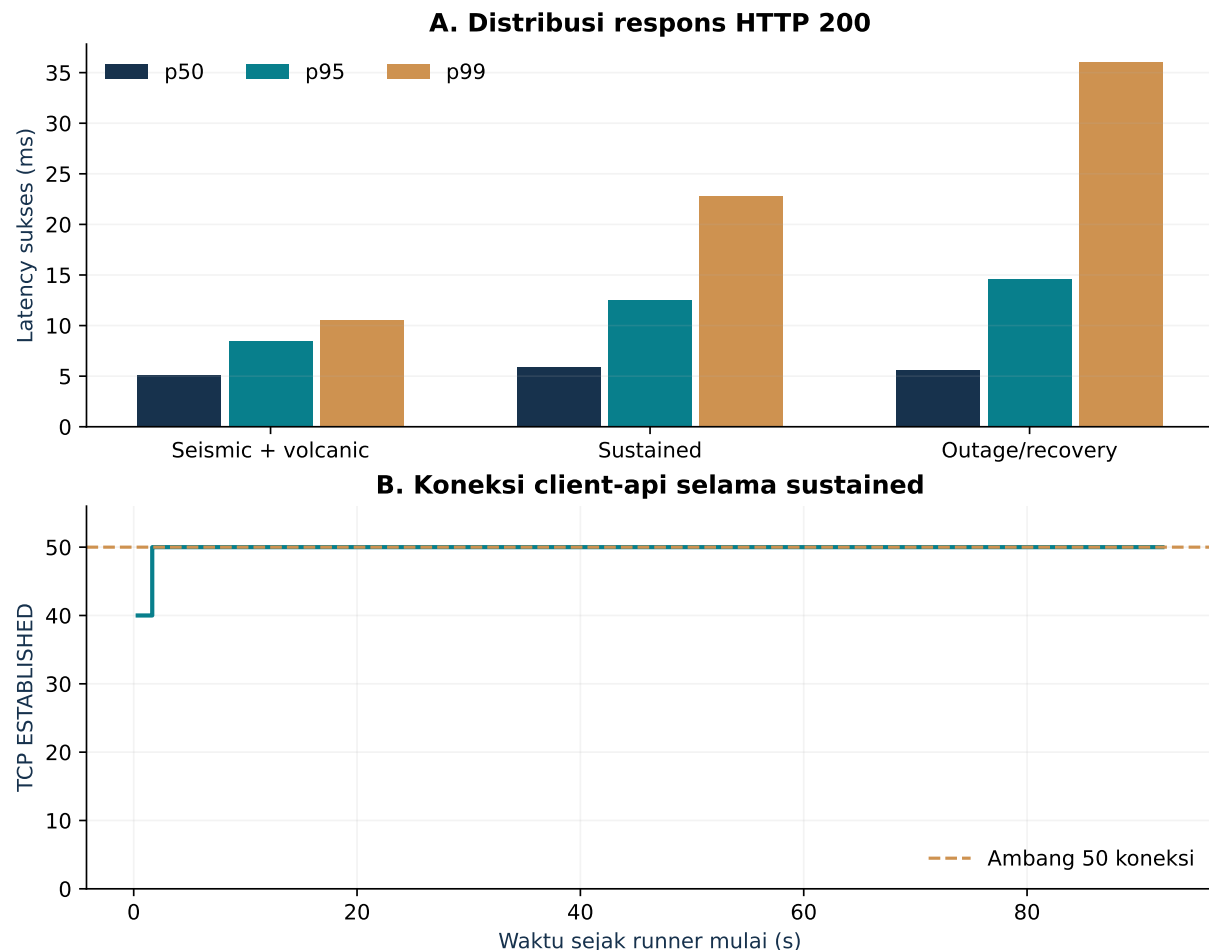
Tabel 2: Hasil akhir k6. Latency hanya HTTP 200; bisnis/s mencakup 429.

Percentile dalam tabel adalah respons HTTP 200 gabungan masing-masing skenario. Untuk kriteria khusus BMKG-only, p95 **seismic saja 8,62 ms**, lebih kecil dari 300 ms ketika pembacaan volcanic berlangsung bersamaan. Durasi mengikuti definisi k6 yang mencakup waktu pengiriman, penantian respons, dan penerimaan data, tetapi tidak mencakup pembentukan koneksi [7]. Penolakan 429 tidak dimasukkan ke distribusi latency sukses.

Pada sustained, 42.619 respons bisnis terdiri atas 9.689 HTTP 200 dan 32.930 HTTP 429. Ada 10 penolakan autentikasi 429 yang dicatat terpisah; auth error setelah mekanisme retry 0%. Throughput sebesar 473,04 respons bisnis per detik mencakup penolakan akibat pembatasan

beban. Apabila hanya respons HTTP 200 yang dihitung, throughput suksesnya sebesar 107,54 respons per detik. Penolakan yang tinggi menunjukkan proteksi bekerja pada pola ini, tetapi bukan bukti bahwa seluruh permintaan diterima.

Error bisnis dihitung sebagai jumlah respons gagal di luar penolakan terkontrol dibagi jumlah respons bisnis selain 429. Pada sustained hasilnya $0/9689 = 0\%$. Sampel 429 dikeluarkan dari pembilang *dan* penyebut agar tidak mengencerkan error rate. Total HTTP bawaan k6 mencampurkan auth, control, dan retry, sehingga bukan metrik throughput bisnis utama.



Gambar 4: Grafik yang dibangkitkan langsung dari JSON hasil k6 dan sampling koneksi. Percentile memakai respons sukses; koneksi diukur terpisah dari jumlah VU.

Runner membaca `/proc/net/tcp` dan `/proc/net/tcp6` sekitar setiap satu detik untuk menghitung koneksi TCP ESTABLISHED pada port client-api. Sedikitnya 50 koneksi teramati terus-menerus selama **90,46 s**. Koneksi aktif secara TCP tidak berarti semua request sedang diproses serentak; jumlah VU saja juga tidak dipakai sebagai bukti koneksi.

Selama outage, 459 dari 474 respons (96,83%) sudah menyatakan sumber bermasalah. Pada fase recovery, 457 dari 727 (62,86%) sudah kembali HEALTHY. Sisanya berada dalam masa deteksi dan pemulihan yang tidak instan. Data tersimpan tetap dilayani dengan HTTP 200. Regresi terpisah memeriksa seismic tetap sehat dan volcanic kembali normal tanpa restart.

Bukti eksekusi tersedia pada [docs/evidence/integration/load/seismic-only.json](#).

Bukti eksekusi tersedia pada [docs/evidence/integration/load/sustained.json](#).

Bukti eksekusi tersedia pada [docs/evidence/integration/load/connections.json](#).

Bukti eksekusi tersedia pada [docs/evidence/integration/load/outage.json](#).

6.5 Validitas hasil dan alternatif

Percobaan awal k6 di container menghasilkan durasi negatif, misalnya minimum -541,10 ms. Walaupun threshold k6 saat itu tidak gagal, hasil tersebut tidak digunakan untuk klaim performa. Artefaknya tetap disimpan pada [docs/evidence/integration/load-docker-invalid/seismic-only.json](#). Penyebab lingkungan belum diisolasi secara pasti. Runner kini menolak nilai durasi negatif, dan seluruh pengukuran final diulang dengan k6 native Windows terhadap stack yang sama.

Alternatif request-time fan-out menjaga kemungkinan kebaruan lebih tinggi, tetapi menempatkan latensi dan kegagalan sumber di jalur client. Materialisasi dipilih karena data historis masih berguna dan P2 menuntut degradasi terkendali. Cache dan micro-cache dapat mengurangi beban query, tetapi menambah invalidasi dan state; belum digunakan. Banyak instance dengan load balancer dapat meningkatkan kapasitas, tetapi memerlukan limit terdistribusi dan koordinasi penulis yang berada di luar M1. Hasil satu mesin ini tidak digeneralisasi sebagai kapasitas produksi.

7 P3. Autentikasi dan Domain Kepercayaan

7.1 Pemisahan kredensial

Domain	Mekanisme	Pemisahan yang ditegakkan
BMKG	Header X-BMKG-Key.	Hanya API key BMKG yang sah.
PVMBG	Authorization: Bearer token PVMBG.	Nilai berbeda dari key BMKG dan JWT downstream.
Admin simulasi	X-Admin-Key.	Kredensial operator terpisah dari akses data.
Downstream	Client ID dan secret berbeda untuk Media, Lapangan, BNPB.	Auth-service menerbitkan scope berdasarkan konfigurasi server.
API internal	X-Internal-Key.	Client-api mengakses Aggregator; bukan token publik client.
Penyimpanan	Password PostgreSQL dan Redis terpisah.	Hanya service pemilik menerima konfigurasi store.

Secret dibangkitkan oleh bootstrap dan dibaca dari environment atau file lokal; private key hanya dipasang pada auth-service dan public key dipasang pada client-api. Berkas contoh memuat placeholder. Mengetahui kredensial Media tidak memberikan kredensial sumber ataupun private key penerbit JWT. Pemisahan ini mempersempit dampak kebocoran satu domain, tetapi bukan pengganti enkripsi transport pada deployment nyata.

7.2 JWT, scope, dan proyeksi

Access token ditandatangani EdDSA dengan key Ed25519 dan TTL default 60 detik. Verifikator membatasi algoritma yang diterima serta memeriksa issuer, audience, expiration, issued-at, subject, ID token, dan scope. Client-api memakai public key lokal, sehingga verifikasi tidak

membutuhkan panggilan auth-service untuk setiap request. Refresh token berupa nilai acak opaque 32 byte; Redis menyimpan hash dan metadata sesi.

Identitas	Hak akses
Media	Ringkasan yang mencakup hazard ID, sumber, jenis, severity, nama area, occurred-at, dan ingested-at.
Tim Lapangan	Ringkasan beserta data mentah yang mencakup source reference ID, koordinat, dan seluruh attributes.
BNPB Pusat	Ringkasan dan mentah; baseline hak tertinggi pada M1 yang tetap read-only.

Permintaan eksplisit field mentah oleh Media ditolak server dengan 403, termasuk raw detail dan `include=raw`. Sesudah query, projector membangun objek baru berdasarkan allowlist. Menambah field upstream dengan demikian tidak otomatis membocorkannya ke Media. Menyembunyikan koordinat hanya pada UI tidak dipakai sebagai mekanisme keamanan.

```

1 func Project(h map[string]any, scope string, fields []string) map[string]any {
2     allowed := append([]string{}, authz.Summary...)
3     if authz.Has(scope, "hazard:read:raw") {
4         allowed = append(allowed, authz.Raw...)
5     }
6     selected := map[string]bool{}
7     for _, f := range fields {
8         selected[f] = true
9     }
10    result := make(map[string]any)
11    for _, f := range allowed {
12        if len(fields) > 0 && !selected[f] {
13            continue
14        }
15        if value, ok := h[f]; ok {
16            result[f] = value
17        }
18    }
19    return result
20 }
```

Listing 2: Proyeksi membangun objek baru dari allowlist, bukan menghapus beberapa field dari objek mentah.

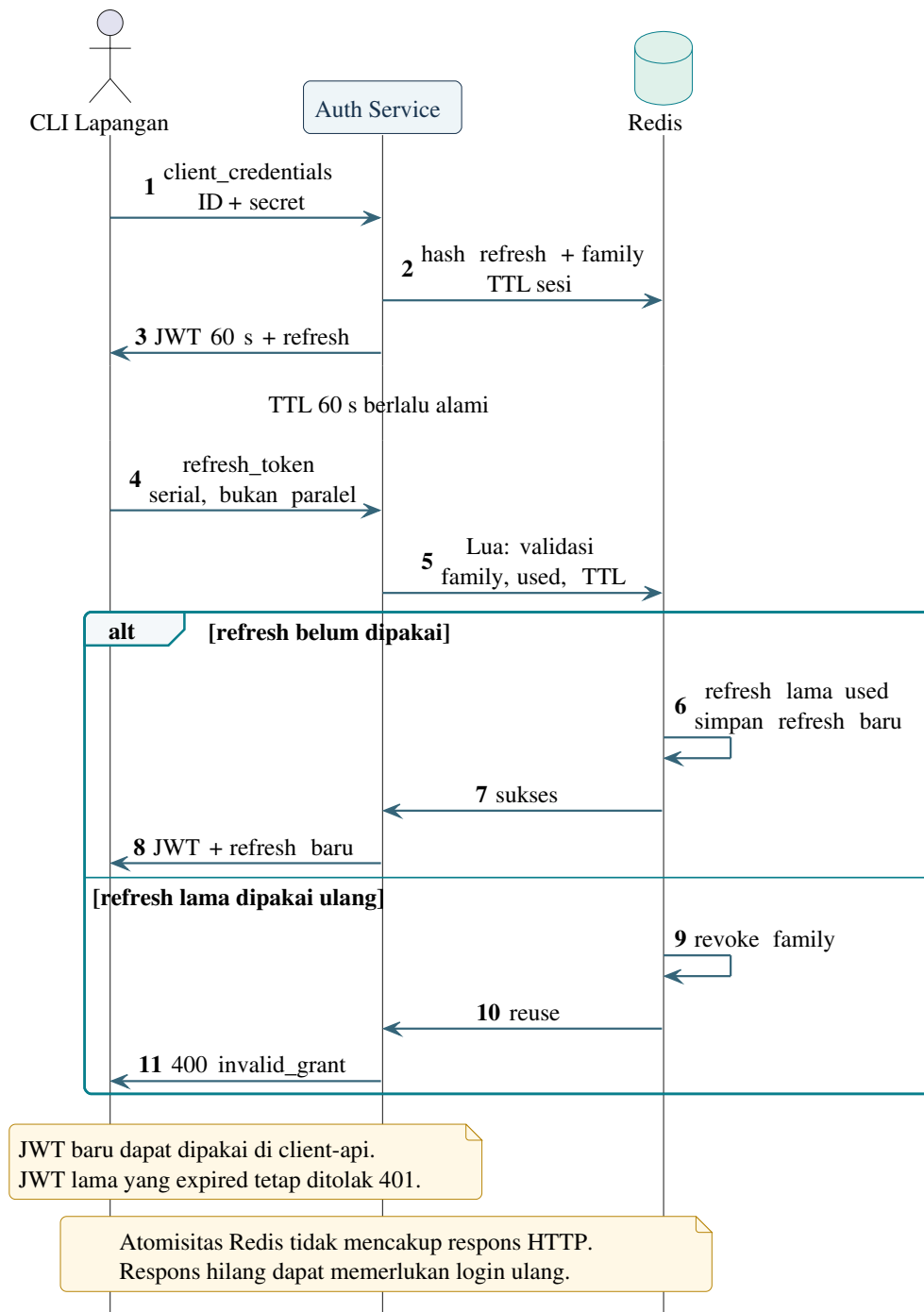
Implementasi terkait terdapat pada [services/client-api/internal/authn/verifier.go](#).

Implementasi terkait terdapat pada [services/client-api/internal/projection/projector.go](#).

7.3 Rotasi refresh dan batas atomisitas

Redis menyimpan keluarga sesi dengan expiry tetap. Lua memvalidasi tipe key, keberadaan, family aktif, kecocokan metadata, dan TTL sebelum mutasi. Pada rotasi normal, refresh lama ditandai used dan refresh baru dibuat dengan sisa masa berlaku yang tidak memperpanjang keluarga. Pemakaian kembali refresh lama mencabut keluarga. Semua langkah tersebut atomik terhadap pengamatan state Redis; script juga menyusun validasi sebelum mutasi karena error runtime Lua tidak otomatis melakukan rollback.

Atomisitas Redis tidak mencakup respons HTTP. Jika rotasi berhasil tetapi respons token baru hilang, retry token lama dapat dianggap reuse; client mungkin harus login ulang. Karena itu satu refresh token tidak digunakan secara paralel oleh CLI maupun antar-VU k6. Ini trade-off proteksi replay yang ketat terhadap kelangsungan sesi pada kegagalan jaringan tertentu.



Gambar 5: Sesi Tim Lapangan, rotasi refresh, serta pemeriksaan token access lama yang sudah kedaluwarsa.

JWT yang sudah diterbitkan diverifikasi offline. Pencabutan refresh family tidak langsung mencabut access token yang masih belum expired. Jendela itu dibatasi TTL pendek; mekanisme introspection atau denylist access token belum diterapkan. Dalam demo expiry, token lama ditolak karena waktu kedaluwarsanya telah lewat, bukan karena refresh secara ajaib membatalkan semua access token lama.

Implementasi terkait terdapat pada [services/auth-service/internal/adapter/outbound/redis/rotate.lua](#).

Implementasi terkait terdapat pada [tools/field-cli/internal/client/client.go](#).

7.4 Alasan pemilihan dan risiko yang ditangani

Satu kredensial bersama antarinstansi ditolak karena menghapus otonomi trust. Session lookup terpusat untuk setiap request menyederhanakan revocation langsung, tetapi menjadikan auth-store dependensi jalur baca. JWT asimetris dipilih agar client-api hanya memiliki kemampuan verifikasi; berbeda dari shared signing secret yang akan memperluas kemampuan penerbitan token.

Validasi algoritma dan claim menghindari penerimaan token untuk issuer atau audience yang salah. Allowlist dan pemeriksaan scope berada di server. Help CLI tidak mencetak secret dari environment; redirect HTTP ditolak untuk mencegah penerusan body login ke tujuan lain. Galat dan trace tidak memuat token atau password. Implementasi ini masih ditujukan untuk PoC lokal. TLS dan mTLS, rotasi key online, MFA, serta pencabutan access token secara langsung belum diimplementasikan. Oleh sebab itu tidak ada klaim bahwa sistem bebas dari seluruh celah keamanan.

7.5 Bukti penyelesaian

Suite fondasi memeriksa kredensial lintas instansi ditolak serta claim JWT invalid tidak diterima. Suite query memverifikasi allowlist Media, penolakan raw, dan akses penuh dua identitas lainnya. `TestNaturalExpiryAndFieldCLI` menjalankan dua request CLI berjarak 65 detik dalam satu sesi dengan default TTL 60 detik, lalu memeriksa token lama 401 sebelum dan sesudah refresh dan token baru 200. Tidak ada manipulasi jam atau TTL singkat buatan untuk bukti ini.

Uji unit CLI memeriksa rotasi serial, respons 401 yang terlambat tidak membatalkan token yang lebih baru, serta penolakan redirect. Catatan fondasi melaporkan Gitleaks 8.24.2 tanpa temuan pada lima commit hingga `f73c035`; cakupannya tidak boleh diperluas menjadi scan revisi akhir. Hasil scan histori final masih perlu dikonfirmasi pada pemeriksaan pengumpulan. Semua nilai contoh pada laporan bebas kredensial nyata.

Bukti eksekusi tersedia pada [docs/evidence/integration/regression.txt](#).

Bukti eksekusi tersedia pada [docs/evidence/integration/modules.txt](#).

Implementasi terkait terdapat pada [docs/evidence/foundation/README.md](#).

8 P4. Batas Service dan Flexible Storage

8.1 Alasan menentukan batas service

BMKG dan PVMBG dipisah karena mewakili instansi otonom, kontrak berbeda, serta kegagalan dan kredensial terpisah. Auth-service menjadi pemilik kebijakan penerbitan token dan sesi; client-api menjadi batas akses downstream. Agregator menjadi pemilik data kanonik, pemetaan, dan transaksi perubahan data. Consumer dipisahkan karena mempunyai tujuan, siklus

hidup, serta state lokal yang berbeda.

Poller, mapper, query internal, dan relay outbox tetap merupakan komponen dalam Aggregator. Memecah setiap komponen itu menjadi service baru akan menambah kontrak jaringan serta mempersulit kepemilikan transaksi tanpa kebutuhan skala M1 yang terbukti. Sebaliknya, menyatukan auth, penyajian client, dan ingest dalam satu deploy akan mengikat perubahan kebijakan akses pada perubahan pemetaan sumber. Pilihan saat ini menerima kompleksitas operasi beberapa container untuk memperoleh perubahan dan recovery yang lebih mandiri.

Setiap service memiliki module Go, entrypoint, Dockerfile, konfigurasi, dan endpoint health. Dependensi shared business package tidak digunakan. Tiga consumer memang memiliki sebagian kode kontrak dan mekanisme delivery serupa; pengulangan kecil ini menjaga build independen pada M1, dengan konsekuensi perubahan envelope perlu diselaraskan dan diuji pada setiap pemakai.

8.2 Kepemilikan Canonical Store

Client-api memanggil `/internal/hazards` dan detail melalui HTTP Aggregator. Ia tidak berada di `store_net` dan tidak menerima kredensial PostgreSQL. Consumer membaca Kafka dan menyimpan view lokal; auth-service memakai Redis. Dengan demikian tidak ada jalur aplikasi biasa yang membaca tabel kanonik secara langsung dari service lain.

Pembatasan ini berlaku pada konfigurasi aplikasi. Test PostgreSQL dan operator yang memiliki Docker dapat mengakses database untuk memeriksa bukti; hak administratif itu tidak diklaim dicegah oleh pemisahan network. Network bukan boundary terhadap administrator mesin.

Implementasi terkait terdapat pada [docker-compose.yml](#).

Implementasi terkait terdapat pada [services/client-api/internal/adapters/outbound/aggregator/client.go](#).

8.3 Skema dan evolusi atribut

Tabel	Fungsi
<code>hazard_events</code>	Field kanonik bertipe, <code>attributes</code> JSONB, keunikan pasangan sumber dan ID sumber, version dan content hash.
<code>outbox</code>	Snapshot perubahan, event ID unik, hazard dan version, dan tanda publish.
<code>checkpoints</code>	Watermark per endpoint yang committed bersama batch.
<code>tsunami_warnings</code>	Korelasi warning sebelum dan sesudah gempa.
<code>source_status</code> dan <code>source_endpoint_status</code>	Status sehat, kegagalan, waktu pengamatan, dan stale.
<code>quarantine</code>	Record yang ditolak beserta alasan dan trace.
<code>schema_migrations</code>	Versi DDL yang dikelola migration runner.

PostgreSQL menyimpan field kanonik yang sering difilter sebagai kolom dengan constraint dan indeks. Atribut yang tidak mempunyai padanan kanonik disimpan sebagai JSONB. Penambahan `confidence_level` dengan demikian merupakan perubahan isi dokumen pada kolom yang sudah ada, bukan penambahan kolom. Migrasi `002_source_status_stale_since` adalah perubahan struktur fitur status sumber, bukan langkah yang dilakukan saat demo penambahan confidence.

JSONB menyediakan representasi JSON yang dapat diproses dan diindeks, tetapi tidak mempertahankan whitespace, urutan key, atau duplicate keys seperti teks asli [5]. Maka “preservasi

atribut” berarti mempertahankan nilai yang didukung, bukan salinan byte-for-byte seluruh wire payload. Indeks utama query saat ini mengikuti waktu kejadian dan hazard ID; tidak ada klaim semua key JSON dinamis otomatis terindeks.

Implementasi terkait terdapat pada [services/aggregator/migrations/001_initial.up.sql](#).

Implementasi terkait terdapat pada [services/aggregator/migrations/002_source_status_stale_since.up.sql](#).

8.4 Perbandingan alternatif penyimpanan

Pilihan	Keunggulan	Konsekuensi
PostgreSQL dengan JSONB (dipilih)	Transaksi hazard, outbox, dan checkpoint; constraint dan query kolom; atribut fleksibel.	Tetap membutuhkan pengelolaan DDL untuk field inti dan kapasitas DB; validasi atribut menjadi tanggung jawab aplikasi.
MongoDB sebagai document store	Dokumen bersarang dan field aditif cocok untuk data heterogen.	Konsistensi lintas record, outbox, indeks, dan operasi deployment tetap harus dirancang; fleksibilitas dokumen tidak otomatis menyelesaikan transaksi.
Relasional kolom tetap saja	Skema dan tipe seluruh atribut sangat eksplisit.	Field sumber baru memerlukan DDL dan koordinasi deploy; bertentangan dengan tujuan perubahan aditif saat runtime.

Perbandingan ini merupakan pertimbangan desain, bukan benchmark database. JSONB dipilih karena kebutuhan transaksi dan query kanonik sudah sesuai dengan PostgreSQL; mengganti store tidak memberikan manfaat M1 yang cukup untuk menambah model konsistensi baru.

8.5 Bukti independensi dan batas

`TestIndependentRebuild` mencatat timestamp start Aggregator, client-api, auth-service, kedua mock, dashboard-updater, dan Kafka; menghentikan notifier; memastikan baca Media masih berhasil; lalu build dan start notifier dengan `-no-deps`. Timestamp komponen lain tidak berubah. Ini bukti konkret satu skenario rebuild independen, bukan klaim semua kombinasi deployment sudah diuji.

`TestDynamicSchemaAndStaleAPI` mengambil record tanpa confidence, mengaktifkan skema baru, membaca record lama dan baru, serta memeriksa versi migrasi dan container tetap. Named volume Canonical Store dan SQLite mempertahankan data pada restart normal. SQLite cocok untuk satu penulis lokal; banyak replica consumer yang berbagi berkas SQLite atau multi-instance Aggregator belum didukung oleh desain koordinasi M1.

Bukti eksekusi tersedia pada [docs/evidence/integration/regression.txt](#).

Implementasi terkait terdapat pada [scripts/check/query_test.go](#).

9 P5. Distribusi Event dengan Consumer Independen

9.1 Broker dan pola yang dipilih

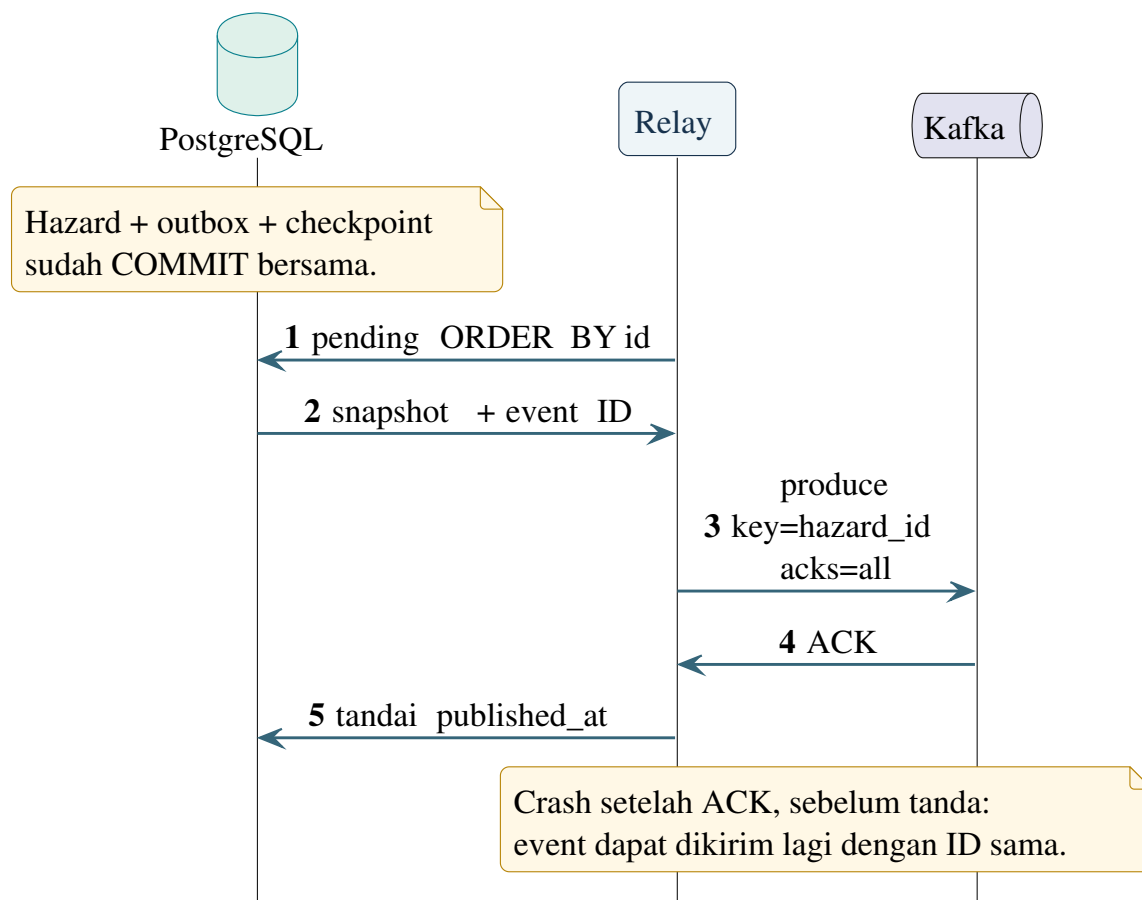
Kafka menyediakan log yang dapat dibaca ulang dan posisi baca per consumer group. Topic utama adalah `bnpb.hazard-events.v1`; kegagalan pesan masuk ke topic DLQ terpisah. Konfigurasi M1 memakai satu partisi, replication factor 1, serta retensi broker tujuh hari. Dashboard-updater, notifier, dan pemda-portal memakai group berbeda sehingga masing-masing memperoleh stream yang sama. Jika mereka memakai satu group, Kafka justru membagi pekerjaan antaranggota; perilaku itu tidak memenuhi kebutuhan tiga proyeksi independen.

Producer memakai key `hazard_id`. Envelope memuat schema version, event ID, event type, hazard ID, version, correlation ID, timestamp, dan snapshot hazard. Bentuk snapshot membuat consumer tidak perlu query balik ke Aggregator saat menangani event. Nama field `published_at` dalam envelope diisi ketika envelope dibuat pada transaksi ingest; waktu ACK publish dicatat pada kolom outbox. Keduanya tidak boleh dianggap timestamp yang identik.

Implementasi terkait terdapat pada [docs/api/hazard-event.md](#).

Implementasi terkait terdapat pada [infra/kafka/init-topics.sh](#).

9.2 Outbox dan semantik pengiriman



Gambar 6: Batas transaksi dan publish. ACK broker mendahului penandaan published pada PostgreSQL; crash di antaranya masih memungkinkan duplikasi.

Menulis database lalu publish langsung dapat kehilangan event ketika proses mati di antaranya. Memublikasikan lebih dahulu dapat menghasilkan event untuk transaksi yang kemudian gagal. Transactional outbox menghindari dua kondisi tersebut dengan menyimpan hazard dan niat publish pada transaksi yang sama. Relay polling membaca pending berurutan berdasarkan ID, mengirim satu record, menunggu ACK, baru menandai baris published. Retensi outbox membersihkan baris yang sudah published lebih dari 24 jam; pending tidak dibersihkan dengan aturan itu.

Producer meminta `acks=all`. Pada replication factor 1, ini tetap hanya ACK satu broker, bukan replikasi tahan kehilangan mesin. Hasil publish yang masih tidak pasti dipertahankan relay agar versi berikutnya tidak melompati pesan sebelumnya. Bila ACK diterima tetapi proses mati sebelum penandaan PostgreSQL, event yang sama dapat dipublikasikan kembali. Karena itu semantik aplikasi adalah **at-least-once**, dengan syarat store dan broker tetap tersedia kembali dan data masih dalam retensi; bukan exactly-once end-to-end.

Urutan Kafka hanya dijamin di dalam partisi [6]. Penambahan partisi dapat mengubah pemetaan key, sehingga klaim urutan per hazard tidak diperluas ke perubahan partisi tanpa strategi transisi. Relay tunggal juga merupakan asumsi; multi-relay memerlukan mekanisme claim dan koordinasi tambahan.

Implementasi terkait terdapat pada `services/aggregator/internal/worker/outbox/relay.go`.

Implementasi terkait terdapat pada `services/aggregator/internal/adapters/outbound/kafka/producer.go`.

9.3 Efek consumer, idempotensi, dan DLQ

Dashboard-updater dan pemda-portal menyimpan view SQLite dengan primary key hazard ID. Upsert hanya mengganti view apabila version masuk lebih besar daripada version tersimpan. Replay atau versi lama tidak menimpa view terbaru. Notifier memeriksa pasangan hazard dan version dan hanya mengirim alert untuk SIAGA atau AWAS; implementasi pengiriman saat ini berupa log simulasi.

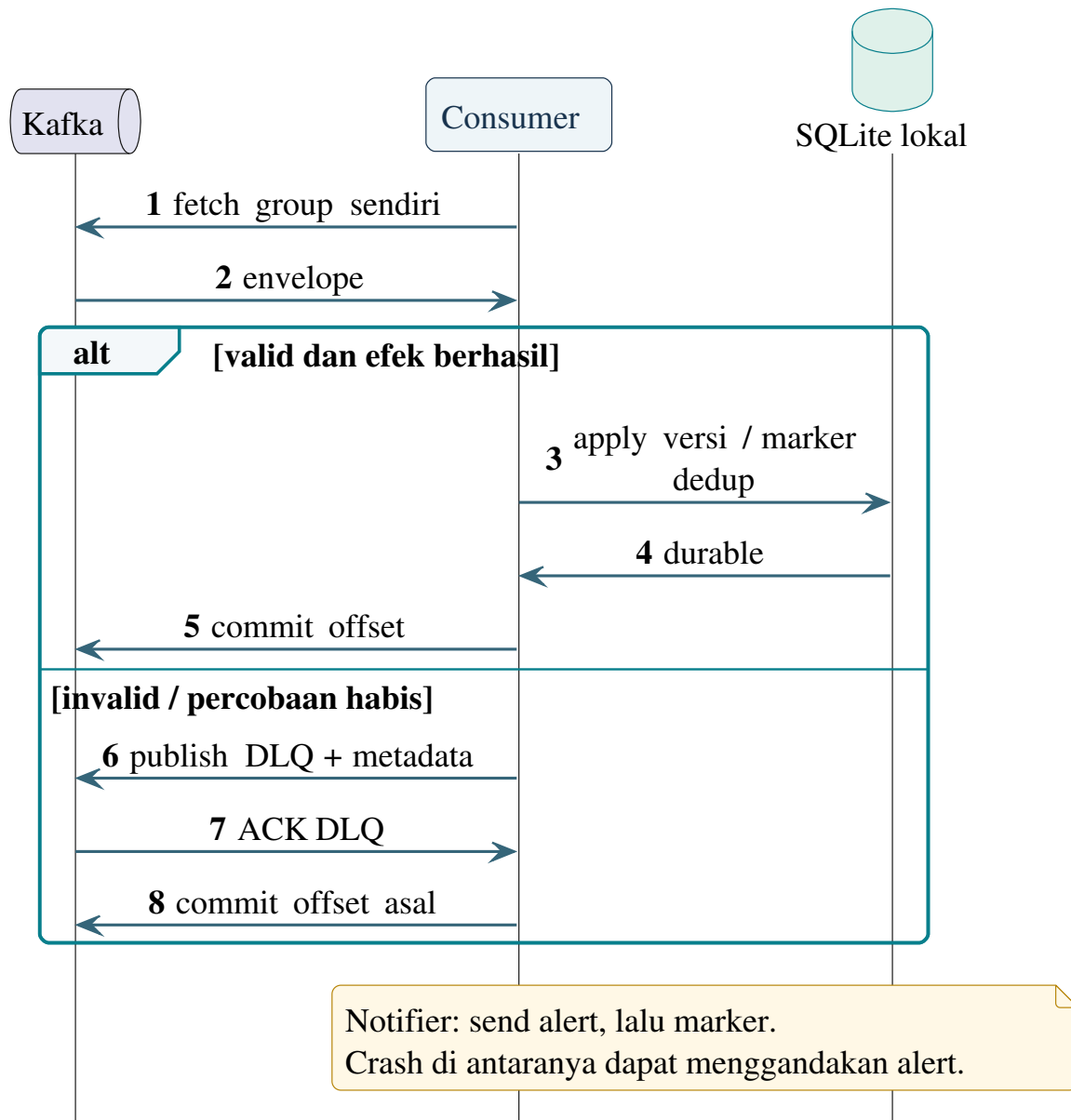
```

1 // Send precedes the durable marker. A crash in between may duplicate delivery.
2 // One sequential consumer owns this SQLite file; it is not a distributed lock.
3 func (s *Service) Handle(ctx context.Context, e contract.Event) error {
4     seen, err := s.Store.Seen(ctx, e.HazardID, e.Version)
5     if err != nil {
6         return err
7     }
8     if seen {
9         return nil
10    }
11    alert := e.Hazard.Severity == "SIAGA" || e.Hazard.Severity == "AWAS"
12    if alert {
13        if err = s.Sender.Send(ctx, e); err != nil {
14            return err
15        }
16    }
17    return s.Store.Record(ctx, e, alert)
18 }

```

Listing 3: Notifier mengirim sebelum mencatat marker persisten; celah crash di antara keduanya diakui sebagai kemungkinan duplikasi.

Consumer menonaktifkan auto-commit. Offset asal di-commit sesudah efek lokal berhasil. Record invalid langsung diarahkan ke DLQ; kegagalan pemrosesan yang dapat dicoba ulang dibatasi oleh jumlah percobaan dan timeout. Setelah percobaan habis, DLQ dipublikasikan dengan identitas group asal, topic, partisi, dan offset, alasan, jumlah percobaan, dan correlation ID. Offset



Gambar 7: Efek lokal atau ACK DLQ mendahului commit offset. Tidak ada transaksi atomik tunggal yang meliputi PostgreSQL, Kafka, dan efek notifikasi.

pesan asal baru di-commit setelah ACK DLQ; kegagalan publish DLQ tidak boleh menghilangkan pesan dengan commit lebih dahulu.

SQLite memakai volume sendiri, WAL, dan synchronous FULL. Efek view persisten serta version guard memberi idempotensi proyeksi. Untuk notifier, crash setelah send tetapi sebelum marker dapat menghasilkan pengiriman ulang. Dedup lokal bukan distributed lock maupun jaminan exactly-once ke perangkat eksternal. Log `record_completed` menunjukkan penanganan record dan commit selesai; record itu bisa berupa duplicate yang diabaikan atau pesan yang masuk DLQ, bukan selalu notifikasi baru.

Implementasi terkait terdapat pada [services/dashboard-updater/internal/store/sqlite.go](#).

Implementasi terkait terdapat pada [services/notifier/internal/application/apply.go](#).

Implementasi terkait terdapat pada [services/notifier/internal/consumer/consumer.go](#).

9.4 Alternatif dan trade-off

Pemanggilan HTTP setiap consumer dari Aggregator lebih sederhana di awal, tetapi mengikat pengetahuan producer tentang subscriber dan membuat kegagalan subscriber masuk jalur pengiriman. RabbitMQ dengan fan-out queue per consumer merupakan alternatif wajar untuk distribusi tugas, namun mekanisme replay histori dan lifecycle queue tetap perlu dirancang. Kafka dipilih untuk retensi log dan offset group; biaya memori, operasional broker, duplikasi, dan pengelolaan retention diterima pada demo lokal. Evaluasi ini tidak mengklaim Kafka selalu lebih unggul pada semua beban.

9.5 Bukti penyelesaian

`TestEventPipeline` memeriksa jalur dari mock melalui outbox dan Kafka hingga dashboard, replay event, penolakan versi lama, dedup notifier, dan persistensi setelah restart. Dashboard dihentikan sementara; notifier dan pemda tetap memproses versi berikutnya. Setelah dashboard dimulai lagi, offset yang belum dikonsumsi tersusul. Ketika Kafka dihentikan, proses ingest tetap menghasilkan outbox dengan status pending. Backlog kemudian terkirim setelah broker kembali tersedia.

Uji subscriber baru menggunakan group pemda unik dan SQLite kosong, sehingga bukti replay tidak bergantung pada state run sebelumnya. Aggregator tidak diganti. Konfigurasi group dan volume pemda reguler dipulihkan setelahnya. Catch-up berlaku selama pesan masih tersedia pada retensi; consumer yang offline melewati retensi tidak dijamin mendapatkan seluruh histori. Replikasi satu broker juga tidak melindungi dari kehilangan seluruh volume Kafka.

Bukti eksekusi tersedia pada [docs/evidence/integration/events-fresh-group.txt](#).

Bukti eksekusi tersedia pada [docs/evidence/integration/event-trace.jsonl](#).

10 Ringkasan Verifikasi dan Batas Hasil

Hasil lokal yang dilaporkan merupakan pengujian bertanggal 6 Oktober 2026. Verifikasi dibagi menjadi unit dan analisis statis dengan go vet, integrasi PostgreSQL, regresi stack, dan load test. Setiap jenis pengujian memberikan bukti dengan cakupan yang berbeda. Uji unit memeriksa logika tertentu, sehingga hasilnya tidak dapat langsung dipakai untuk menyimpulkan perilaku deployment, ketahanan mesin, atau kapasitas produksi.

Pemeriksaan	Hasil dan cakupan
Sembilan module Go	Bootstrap, unit test, dan vet; delapan service serta CLI.
PostgreSQL nyata	Transaksi ingest dan query lulus dalam schema test terisolasi.
Regresi stack	Fondasi, ingest, event, query, expiry token alami, serta rebuild independen lulus; durasi run 234,280 s.
Fresh subscriber	Group baru dan store kosong berhasil replay; suite event tambahan lulus dalam 71,575 s.
Readiness consumer	Test broker tidak merespons memenuhi deadline 50 ms dan menolak penumpukan probe.
Load final	Tiga skenario lulus; angka dan batas pengukuran pada Bagian 6.
GitHub Actions	Workflow tersedia; hasil hosted belum dapat diverifikasi dari sesi penyusunan laporan.

Kegagalan awal tidak dihapus dari bukti. Regresi pertama menemukan timeout readiness consumer saat Kafka mati. Pengujian terisolasi menunjukkan Ping library dapat menunggu handshake sekitar 10 s meskipun context sudah deadline. Perbaikan membatasi waktu tunggu caller dan satu probe yang belum selesai; regresi penuh kemudian lulus. Hasil gagal pertama tersedia pada [docs/evidence/integration/regression-before-readiness-fix.txt](#), sedangkan hasil akhir pada [docs/evidence/integration/regression.txt](#).

```

--- PASS: TestHealth (0.04s)
--- PASS: TestMockContracts (3.92s)
--- PASS: TestPVMBSchemaAndOutage (3.09s)
--- PASS: TestTokenRotationAndAuthorization (0.11s)
--- PASS: TestJWTClaimValidation (0.01s)
--- PASS: TestPersistenceAndDependencyRecovery (18.73s)
--- PASS: TestStructuredLogsDoNotExposeCredentials (0.42s)
--- PASS: TestIngestPipeline (44.77s)
--- PASS: TestEventPipeline (53.22s)
--- PASS: TestQueryIntegration (0.53s)
--- PASS: TestDynamicSchemaAndStaleAPI (21.27s)
--- PASS: TestNaturalExpiryAndFieldCLI (68.92s)
--- PASS: TestIndependentRebuild (18.32s)
PASS
ok      command-line-arguments  234.280s

```

Listing 4: Petikan keluaran pengujian aktual. Ini transkrip test, bukan screenshot yang disimulasikan.

Perlu dilengkapi. Status GitHub Actions dan tautan run final: _____ . API tanpa autentikasi pada sesi penyusunan mengembalikan 404, sehingga keberhasilan CI hosted tidak diasumsikan.

10.1 Kesimpulan dan batas cakupan

Alur ingest, query, otorisasi, serta distribusi event telah terhubung dalam implementasi. Bukti pengujian lokal mendukung skenario P1 hingga P5 yang dipetakan pada lampiran. Cakupan sistem masih terbatas pada PoC dalam satu host dan belum mencakup failover lintas node, koordinasi multi-instance Agregator, limit terdistribusi, transport terenkripsi end-to-end, maupun pengiriman notifikasi nyata ke perangkat.

Demo sinkron tetap perlu dijalankan kelompok dan tidak digantikan oleh laporan test. Verifikasi panjang outage 10 hingga 20 menit secara langsung belum direpresentasikan oleh load test 20 detik yang disimpan; endpoint operator mendukung outage hingga dimatikan, tetapi kemam-

puan konfigurasi tidak sama dengan bukti durasi demo tertentu. Tag final dan pengumpulan juga merupakan tindakan terpisah setelah kelompok meninjau laporan.

11 Deklarasi Penggunaan LLM

Kelompok menggunakan Claude untuk membantu menyusun rancangan awal dan OpenAI Codex untuk membantu peninjauan, implementasi dan perbaikan, penyusunan pengujian, dokumentasi, serta penyusunan sumber LaTeX laporan ini. Rancangan awal tidak diperlakukan sebagai bukti bahwa seluruh mekanismenya telah dibangun; narasi laporan diselaraskan dengan source code dan artefak eksekusi yang tersedia.

Angka performa berasal dari hasil k6 dan pengukuran koneksi yang disimpan di repository. Diagram merupakan representasi yang dibuat dari implementasi, bukan tangkapan layar aplikasi. Potongan kode dan transkrip test diambil dari berkas asli. Tidak ada hasil pengujian, identitas anggota, atau pembagian kontribusi yang diisi berdasarkan dugaan.

Perlu dilengkapi. Lengkapi rincian tambahan penggunaan LLM oleh masing-masing anggota bila ada, termasuk bantuan pada query dan API internal Aggregator, Client API, CLI Tim Lapangan, dan script k6 yang belum dinyatakan. Kelompok wajib meninjau isi dan memastikan seluruh kode serta keputusan yang dikumpulkan dipahami sebelum menyerahkan laporan.

A Matriks Kriteria Spesifikasi

ID di bawah mengikuti urutan kriteria pada spesifikasi M1. “Lokal” berarti telah diperiksa oleh suite yang dirujuk, bukan pengganti pelaksanaan demo sinkron. Rujukan U1 hingga U7 mengikuti ketentuan implementasi umum. Indeks bukti E1 hingga E8 dijelaskan pada Lampiran B.

ID	Kriteria dan mekanisme	Verifikasi dan bagian laporan
U1	Panggilan jaringan; tidak berbagi business logic lintas service.	Module terpisah, Compose, dan kode adapter; Bagian 2.
U2	Satu pemilik storage; client-api melalui API Aggregator.	Konfigurasi network pada Compose beserta adapter HTTP; Bagian 8.
U3	Dockerfile per service dan satu orkestrasi.	Dockerfile delapan service, Compose; E8 dan petunjuk start.
U4	Kegagalan dan delay benar-benar dipicu.	Outage PVMBG dan Kafka, restart dependensi, rebuild notifier; E2, E3, dan E4.
U5	Kredensial sumber tidak berlaku lintas instansi.	TestMockContracts ; E2.
U6	Secret dari konfigurasi lokal dan contoh environment.	Generator secret, ignore, .env.example ; Bagian 7.
U7	Health, log terstruktur, correlation ID, dan latency.	Handler, log outbound, poller, relay, dan consumer, E6 dan E8. Latency trace yang ditampilkan bukan distribusi seluruh operasi internal DB.
P1.1	Pemetaan kanonik kedua sumber.	Unit mapper dan TestIngestPipeline ; E1 dan E2.
P1.2	Schema v2 dipicu saat runtime.	Endpoint admin dalam TestDynamicSchemaAndStaleAPI ; E2.
P1.3	Ingest tetap valid tanpa perubahan kode atau restart.	Record baru terbaca; ID container dan versi DDL tetap; E2.
P1.4	Kebijakan unknown field dinyatakan dan konsisten.	Extra ke attributes, validasi dan karantina; Bagian 5, E1 dan E2.
P1.5	Mekanisme serta perubahan yang didukung dan yang tidak didukung.	Tolerant reader additive-only dan batas tipe; Bagian 5.
P2.1	Seismic p95 < 300 ms saat volcanic paralel dan delay 3 s.	8,62 ms; E4.
P2.2	Minimal 50 koneksi, minimal 60 s, error < 1% di luar 429.	50 TCP selama 90,46 s; error 0%; 32.930 bisnis 429 dilaporkan; E4 dan E5.
P2.3	Seismic tetap dilayani, volcanic stale atau unavailable, lalu recovery.	Status stale dan pemulihan tanpa restart; E2 dan E4.
P2.4	Model konkurensi, pencegahan tiga kegagalan, konfigurasi load.	Bagian 6; script k6 dan runner.
P3.1	BMKG dan PVMBG menolak kredensial silang.	HTTP 401 atau 403 pada suite fondasi; E2.
P3.2	Media hanya ringkasan dan raw eksplisit ditolak.	TestQueryIntegration ; E2.
P3.3	Expiry alami, refresh otomatis, token lama ditolak.	TTL 60 s, sesi CLI 65 s, pemeriksaan 401 dan 200; E2.

ID	Kriteria dan mekanisme	Verifikasi dan bagian laporan
P3.4	Tidak ada secret yang ditulis langsung dalam kode atau tersimpan dalam commit.	Bootstrap, contoh env, ignore, test log; scan fondasi terbatas, scan final belum terkonfirmasi; Bagian 7.
P3.5	Alasan keamanan mekanisme dan batasnya.	Domain trust, claim, allowlist, replay, transport, dan revocation; Bagian 7.
P4.1	Penghentian, build ulang, dan pengaktifan kembali satu komponen tanpa deploy yang lain.	Notifier rebuild, baca API tetap berhasil, timestamp start lain tetap; E2.
P4.2	Storage fleksibel tanpa DDL untuk field baru; record lama dan baru tersedia bersama.	Query record sebelum dan sesudah confidence; E2.
P4.3	Canonical Store hanya diakses aplikasi pemilik.	Network, kepemilikan secret, dan query melalui HTTP; Bagian 8.
P4.4	Alasan batas service dan minimal dua alternatif storage.	Perbandingan PostgreSQL dengan JSONB, document store, dan kolom tetap; Bagian 8.
P5.1	Hazard baru dikirim ke broker.	Commit outbox, publish ACK, dan konsumsi snapshot; E3.
P5.2	Minimal dua consumer independen.	Dashboard dan notifier dengan group terpisah; E3.
P5.3	Satu consumer mati tidak menahan yang lain; jelaskan catch-up.	Dashboard dihentikan dan diaktifkan kembali, versi susulan, retensi; E3.
P5.4	Subscriber baru tanpa perubahan producer.	Group pemda unik dan store kosong; Agregator tidak diganti; E3.
P5.5	Broker, pola, semantik pengiriman, dan idempotensi.	At-least-once, version guard, marker, batas notifier; Bagian 9.

Kelengkapan laporan dan pengumpulan

Sampul, kontribusi, deskripsi, diagram beserta narasi, teknologi, asumsi, petunjuk menjalankan, dan pembahasan per problem tersedia. Kontribusi penulisan serta rincian tambahan penggunaan LLM masih berupa placeholder. PDF laporan menggunakan nama berkas yang diminta, yaitu `IF4031_M1_AkuAnakTehat.pdf`.

Perlu dilengkapi. Sebelum pengumpulan, lengkapi placeholder kontribusi dan deklarasi, periksa hasil CI dari akun yang memiliki akses, gladi demo, tentukan commit final dan tag `milestone-1`, serta isi formulir kelompok. Tautan PDF final:

B Indeks Bukti dan Jejak Eksekusi

Seluruh berkas E1 hingga E8 berada di `docs/evidence/integration/` pada revisi acuan. Artefak disimpan sebagai teks dan JSON agar dapat ditinjau tanpa bergantung pada screenshot. Potongan yang dicetak merupakan petikan atau rekam berlabel; output lengkap tetap menjadi acuan.

ID	Berkas	Isi
E1	<code>docs/evidence/integration/modules.txt</code>	Uji unit dan go vet sembilan module dan bootstrap.
E2	<code>docs/evidence/integration/regression.txt</code>	Regresi lengkap termasuk auth, ingest, query, drift, dan rebuild.
E3	<code>docs/evidence/integration/events-fresh-group.txt</code>	Replay subscriber baru, DLQ, dedup, dan outage broker.
E4	<code>docs/evidence/integration/load/seismic-only.json</code> , <code>docs/evidence/integration/load/sustained.json</code> , <code>docs/evidence/integration/load/outage.json</code>	Summary k6 final, terpisah dari hasil timer invalid.
E5	<code>docs/evidence/integration/load/connections.json</code>	Sampel waktu host dan koneksi TCP teramati.
E6	<code>docs/evidence/integration/event-trace.jsonl</code>	Trace nyata dari mock sampai consumer.
E7	<code>docs/evidence/integration/postgres.txt</code>	Integrasi transaksi dan query PostgreSQL.
E8	<code>docs/evidence/integration/services.txt</code>	Snapshot status container akhir sesi pengujian.

B.1 Contoh trace event nyata

Correlation ID: 2a877ebd-3298-4ba1-8f24-8bed0d80d6e3.

Service	Pesan log	Latency (ms)
bmkg-mock	http_request	100.426
aggregator	poll_complete	118
aggregator	outbox_published	1
pemda-portal	record_completed	6
notifier	notification_simulated	–
dashboard-updater	record_completed	5
notifier	record_completed	4

Correlation ID dibentuk pada polling lalu diteruskan ke mock, transaksi dan envelope, publish, dan consumer. Query client memiliki correlation ID tersendiri; query tidak dianggap sebagai penyebab ingest. Pada log consumer, latency menunjukkan waktu pemrosesan record sampai commit, bukan latency ujung-ke-ujung sejak kejadian bencana.

B.2 Asal potongan kode dan gambar

Potongan kode di `assets/code/` diekstrak dari revisi acuan, dengan lokasi dan hash pada `assets/provenance.json`. Sumber enam diagram berada di `diagrams/`; grafik performa dibangkitkan dari JSON bukti final. Semua dapat dibuat ulang dengan `scripts/refresh-assets.py`. Gambar bukan hasil tangkapan layar implementasi atau bukti tambahan yang dibuat-buat.

B.3 Dokumentasi demonstrasi

Perlu dilengkapi. Tanggal dan tautan rekaman atau foto demonstrasi sinkron, bila diminta: _____. Hasil test lokal dalam laporan tidak dinyatakan sebagai demonstrasi sinkron yang sudah berlangsung.

Pustaka

- [1] Tim pengampu IF4031. *Tugas Besar – Milestone 1*, berkas yang diberikan kelompok: IF4031 - Tugas Besar - M1 (1).pdf, 29 halaman, 2026. Bagian spesifikasi, problem set, dan laporan, hlm. 4–29.
- [2] Tim pengampu IF4031. *Tugas Besar – Dokumen Terpusat*, berkas yang diberikan kelompok: IF4031 - Tugas Besar - Dokumen Terpusat (1).pdf, 10 halaman, 2026.
- [3] Kelompok AkuAnakTehat. *Rancangan Arsitektur BNPB*, dokumen kerja awal yang disusun dengan bantuan Claude, September 2026. Digunakan sebagai bahan alasan desain; klaim implementasi diverifikasi kembali.
- [4] Kelompok AkuAnakTehat. *Repository implementasi dan bukti M1*, revisi f6e99cc. <https://github.com/WwzFwz/AkuAnakTehat/tree/f6e99cc10fec64042eb184c9d7f92d14cb22fd23>.
- [5] PostgreSQL Global Development Group. *PostgreSQL 16 Documentation: JSON Types*. <https://www.postgresql.org/docs/16/datatype-json.html>. Diakses 8 Oktober 2026.
- [6] Apache Kafka. *Kafka 3.9 Design: Message Delivery Semantics*. <https://kafka.apache.org/39/design/design/>. Diakses 8 Oktober 2026.
- [7] Grafana Labs. *k6 Built-in Metrics*. <https://grafana.com/docs/k6/latest/using-k6/metrics/reference/>. Definisi metrik dibaca bersama script k6 versi 0.57.0 yang digunakan dalam pengujian.